wait up to the timeout period for matching elements to be found, and will fail the test if no matching element is found at the end of the timeout period.

Cypress works asynchronously, so you can't return the results of a finder method to a variable; you typically continue to chain method calls that send actions or assertions.

That said, if you want to hold on to a DOM element or elements that you've located, you can use the `as` command to store those elements as an alias, as we've seen with `cy.get(".concert").first().as("concert")`. You can access that alias later in the test with `cy.get("@concert")`. Aliases are not limited to DOM elements; you can use them to store other data, and aliases that you define in a `beforeEach` method are accessible in the tests that use that method.

Actions

Once you've identified the DOM element or elements you want to interact with, you can then send actions to those elements. Our first test used `click` to interact with a DOM element on the page and send it an action. Cypress provides basically any action you'd want:

- Click actions: `click`, `dblclick`, `rightclick`
- Form actions: `check`, `clear`, `select`, `submit`, `type`, `uncheck`
- Focus actions: `blur`, `focus`
- Scroll actions: `scrollIntoView`, `scrollTo`
- Send a generic event: `trigger`

All of these are basically the names of their associated actions. The ones that might not be forthcoming are `type`, which places text into a text field, and `clear`, which clears a text field. You use all of them by chaining onto a set of commands that yields DOM elements.

Cypress Has No Hover